

AAII - Tema 1: Métodos de ensamble paralelos

Este tema se llamaba originalmente “Bosques aleatorios”, pero se consideran alternativas como “Combinación paralela de modelos” o “Métodos de ensamble paralelos” para que englobe más genéricamente los distintos temas tratados.

Se comienza revisando el concepto de la descomposición sesgo-varianza para comprender por qué combinar modelos inestables, como los árboles de decisión, puede mejorar la capacidad de generalización.

Después se repasa bootstrap, como técnica de remuestreo aleatorio con reemplazo utilizada para construir ensembles como bagging y bosque aleatorio.

Se continúa revisando las siguientes estrategias generales de ensamble, que son aplicables en el contexto de ensembles basados en árboles:

- Intensificación (*boosting*)
- Bagging
- Subespacio aleatorio (*random subspace*)
- Subespacios mediante selección de características (*feature selection subspace*)
- Bagging mediante transformaciones de datos (*data transform bagging*).

Por último, se describen los siguientes métodos ensamble basados en árboles:

- Bosque aleatorio
- ExtraTree

Descomposición en sesgo/varianza

Bibliografía:

- [ISL] 2.2.2.
- [ESL] 7.1-7.3. Págs. 219-228
- Diagrama sesgo varianza
- [URF] 4.1.1 “Bias-variance decomposition” Págs. 55-60

En aprendizaje estadístico se distinguen entre los distintos tipos de errores:

- **Error de entrenamiento:** error cometido sobre los mismos datos usados para ajustar el modelo.
- **Error de test o error de generalización:** error cometido sobre datos nuevos, no usados para entrenar.

EXAM=(2026J2.3)

El error de entrenamiento suele ser demasiado optimista, porque se calcula sobre los mismos datos que el modelo ha usado para aprender. Por eso, normalmente será menor que el error de test, que se evalúa sobre datos nuevos no vistos durante el entrenamiento.

Lo importante en aprendizaje automático no es únicamente acertar en entrenamiento, sino generalizar bien.

Un modelo más complejo o flexible suele poder reducir mucho su error de entrenamiento. Sin embargo, eso no garantiza que reduzca su error de test.

- Al aumentar la flexibilidad, el error de entrenamiento normalmente disminuye.
- El error de test suele disminuir al principio, pero puede volver a crecer si el modelo se adapta demasiado al ruido de entrenamiento.
- Esta forma típica del error de test es una curva en U.

Se habla de **sobreajuste** u ***overfitting*** cuando un modelo se ajusta excesivamente a los datos de entrenamiento y empeora en datos nuevos.

EXAM=(2022SO.3)

En regresión, usando como función de pérdida el error cuadrático, el error esperado de predicción puede descomponerse así:

$$\text{Error esperado} = \text{Error irreducible} + \text{Sesgo}^2 + \text{Varianza}$$

También puede aparecer escrito como:

$$E[(Y - \hat{f}(x))^2] = \text{Var}(\varepsilon) + [\text{Bias}(\hat{f}(x))]^2 + \text{Var}(\hat{f}(x))$$

El **error irreducible** es la parte del error debida al ruido propio de los datos.

Podemos imaginar que la salida real sigue una relación ideal:

$$Y = f(X) + \varepsilon$$

donde:

- $f(X)$ es la relación real que querríamos aprender;
- ε representa ruido, errores de medida, información no observada o variabilidad inevitable.

Dada la fórmula, el error irreducible es la varianza de ese ruido, $\text{Var}(\varepsilon)$.

Ideas clave sobre el error irreducible:

- No depende del modelo elegido.
- No desaparece aunque tengamos un modelo perfecto.
- Es el límite inferior del error esperado: ningún modelo puede obtener sistemáticamente un error inferior al ruido inevitable del problema.

Por ejemplo, si queremos predecir el tiempo que tarda una persona en terminar una tarea, puede haber factores que no están en los datos: cansancio, interrupciones, errores de medición, etc. Aunque el modelo sea muy bueno, ese ruido seguirá existiendo.

El **sesgo** mide cuánto se equivoca sistemáticamente el modelo por usar una representación demasiado simple del problema.

Dicho de forma intuitiva, un modelo tiene alto sesgo cuando, aunque lo entrenemos muchas veces con distintos datos, suele equivocarse en la misma dirección porque su forma de representar el problema es insuficiente.

Un ejemplo es que si la relación real entre entrada y salida es muy curva y usamos siempre una recta, la recta no podrá representar correctamente el patrón real. Aunque añadamos muchos datos, el problema seguirá existiendo: el modelo es demasiado simple.

Alto sesgo suele asociarse con

- Modelos poco flexibles.
- Relaciones demasiado simplificadas.
- Subajuste o *underfitting*.
- Error alto tanto en entrenamiento como en test.

Como conclusión, un modelo simple suele tener sesgo alto y varianza baja.

La **varianza** mide cuánto cambiarían las predicciones del modelo si lo entrenásemos con conjuntos de datos diferentes.

Dicho de forma intuitiva, un modelo tiene alta varianza cuando pequeños cambios en los datos de entrenamiento producen modelos o predicciones muy diferentes.

Un ejemplo sería un modelo extremadamente flexible que puede adaptarse a cada detalle y a cada ruido concreto del conjunto de entrenamiento. Si se cambian unas pocas observaciones, el modelo aprendido puede cambiar bastante.

Alta varianza suele asociarse con:

- Modelos muy flexibles.
- Mucha sensibilidad a los datos concretos de entrenamiento.
- Sobreajuste u *overfitting*.
- Error de entrenamiento muy bajo, pero error de test elevado.

Como conclusión, un modelo muy complejo suele tener Sesgo bajo y varianza alta.

Generalmente no se puede reducir simultáneamente sesgo y varianza sin límite.

Al aumentar la flexibilidad del modelo:

Flexibilidad del modelo	Sesgo	Varianza	Riesgo típico
Baja	Alto	Baja	Subajuste
Intermedia	Moderado	Moderada	Mejor generalización
Alta	Bajo	Alta	Sobreajuste

Con un modelo demasiado simple, el sesgo es alto y la varianza es baja, y el error de test es alto porque el modelo no captura bien el patrón.

Al aumentar moderadamente la flexibilidad:

- baja bastante el sesgo;
- la varianza todavía no crece demasiado;
- el error de test disminuye.

Si aumentamos demasiado la flexibilidad:

- el sesgo puede bajar poco más;
- la varianza aumenta mucho;
- el error de test vuelve a subir.

En conclusión, el mejor modelo no es necesariamente el que mejor memoriza entrenamiento, sino el que consigue un buen equilibrio entre sesgo y varianza para predecir datos nuevos.

Situación	Qué ocurre	Sesgo	Varianza
Subajuste (<i>underfitting</i>)	El modelo es demasiado simple y no aprende bien el patrón	Alto	Baja
Buen ajuste	El modelo captura el patrón sin seguir demasiado el ruido	Moderado/bajo	Moderada/baja
Sobreajuste (<i>overfitting</i>)	El modelo aprende detalles y ruido de entrenamiento	Bajo	Alta

Pistas típicas:

- “Modelo demasiado simple” → **alto sesgo**.
- “Modelo cambia mucho si cambian los datos de entrenamiento” → **alta varianza**.
- “Muy buen resultado en entrenamiento pero malo en test” → **sobreajuste / alta varianza**.

- “No consigue representar una relación compleja” → **subajuste / alto sesgo**.

La fórmula anterior se presenta de forma directa para **regresión con error cuadrático**.

En **clasificación**, especialmente si se mide el error como acierto/fallo, la relación es menos directa:

- una predicción de probabilidad puede estar sesgada;
- pero mientras siga dando la clase correcta, no produce error de clasificación.

Ejemplo sencillo:

Si la probabilidad real de una clase es 0.9 y el modelo estima 0.6, la estimación no es perfecta, pero ambas conducen a predecir la misma clase. Por tanto, puede haber sesgo sin que aumente inmediatamente el número de fallos de clasificación.

La descomposición aditiva exacta error irreducible + sesgo² + varianza es la referencia fundamental en regresión con pérdida cuadrática. En clasificación, sesgo y varianza siguen siendo ideas útiles, pero su efecto sobre el error de clasificación no es tan directo.

Los árboles de decisión pueden llegar a ser modelos muy flexibles. Esa flexibilidad permite capturar relaciones complejas, pero también puede hacer que sean sensibles a los datos usados para entrenarlos.

Una de las motivaciones de los métodos basados en conjuntos de árboles será controlar el error de generalización actuando especialmente sobre la varianza.

El método bootstrap

Bibliografía:

- [WIB]
- [ISL 5.2]

EXAM=(AAII.EX.2022S0.2)

El **bootstrap** es una técnica estadística de remuestreo aleatorio con reemplazo.

Se parte de una única muestra observada y se generan muchas muestras nuevas, llamadas **muestras bootstrap**, tomando observaciones de la muestra original aleatoriamente, con reemplazo y manteniendo normalmente el mismo tamaño que la muestra original.

La finalidad de bootstrap es aproximar qué habría ocurrido si hubiéramos podido obtener muchas muestras diferentes de la población real. Se emplea para estimar

parámetros de una población y, especialmente, cuantificar la incertidumbre o variabilidad de sus estimaciones.

Cuando se dice **muestra con reemplazo** o **con repetición**, se refiere a que, después de seleccionar una observación, esta se devuelve al conjunto y puede volver a seleccionarse. Por estas razones, una muestra bootstrap puede contener valores repetidos y omitir otros, mientras que el tamaño de la muestra bootstrap suele ser el mismo que el de la muestra original..

Ejemplo: si la muestra original es:

2, 4, 5, 6, 6

algunas muestras bootstrap posibles son:

- 2, 5, 5, 6, 6
- 4, 5, 6, 6, 6
- 2, 2, 4, 5, 5
- 2, 2, 2, 2, 2

En estadística normalmente queremos conocer algún valor de una población completa, por ejemplo:

- la media del peso de todos los productos fabricados;
- la proporción de usuarios que comprarían un producto;
- la variabilidad de un coeficiente estimado por un modelo;
- el error estándar de una estimación.

El problema es que casi nunca podemos observar toda la población. Solo disponemos de una **muestra**.

A partir de esa muestra calculamos un valor, llamado **estadístico**, que utilizamos como estimación del parámetro poblacional:

Concepto	Ejemplo
Parámetro poblacional	Media real de todos los productos
Estadístico muestral	Media calculada con los productos observados
Incertidumbre	Cuánto podría cambiar esa media si hubiéramos tomado otra muestra

El bootstrap permite estimar la incertidumbre derivada del estadístico muestral reutilizando de forma controlada la muestra que ya tenemos.

El procedimiento bootstrap es:

1. Tomar aleatoriamente n observaciones de la muestra original **con reemplazo**.
2. Formar con ellas una nueva muestra bootstrap.
3. Calcular en esa muestra el estadístico que nos interesa: media, proporción, coeficiente, etc.
4. Repetir el proceso muchas veces, por ejemplo $B = 1000$ veces.
5. Analizar la distribución de los B valores obtenidos.

Si el estadístico calculado cambia mucho entre remuestras bootstrap, la estimación original es poco estable. Si cambia poco, la estimación es más precisa.

Lo ideal para estudiar la precisión de una estimación sería:

1. Obtener muchas muestras independientes de la población real.
2. Calcular el estadístico en cada muestra.
3. Ver cuánto varían los resultados.

Normalmente eso no es posible, porque:

- obtener nuevos datos puede ser caro;
- la población completa puede ser inaccesible;
- solo disponemos de la muestra recogida originalmente.

Bootstrap sustituye esas nuevas muestras reales por remuestras generadas a partir de la muestra observada.

Sea una muestra original:

$$Z = \{z_1, z_2, \dots, z_n\}$$

A partir de ella generamos B muestras bootstrap:

$$Z^{*1}, Z^{*2}, \dots, Z^{*B}$$

En cada muestra calculamos el estadístico que nos interesa. Por ejemplo, si queremos estudiar una estimación $\hat{\theta}$, obtenemos:

$$\hat{\theta}^{*1}, \hat{\theta}^{*2}, \dots, \hat{\theta}^{*B}$$

La variabilidad de estos valores sirve para estimar la incertidumbre de $\hat{\theta}$.

Una aplicación importante del bootstrap es estimar el **error estándar** de un estimador.

El error estándar indica cuánto variaría aproximadamente una estimación si pudiéramos repetir el muestreo muchas veces.

Si tenemos B estimaciones bootstrap, su media es:

$$\overline{\hat{\theta}^*} = \frac{1}{B} \sum_{r=1}^B \hat{\theta}^{*r}$$

El error estándar bootstrap puede estimarse mediante:

$$SE_B(\hat{\theta}) = \sqrt{\frac{1}{B-1} \sum_{r=1}^B (\hat{\theta}^{*r} - \overline{\hat{\theta}^*})^2}$$

Se observa cuánto varían entre sí las estimaciones obtenidas en las distintas muestras bootstrap.

Supongamos que tenemos los pesos de 100 productos y calculamos que la media es 50 gramos.

Solo con esa media no sabemos si la estimación es muy estable o si cambiaría bastante tomando otros 100 productos.

Con bootstrap:

1. Generamos muchas muestras de tamaño 100, tomando productos de la muestra original con reemplazo.
2. Calculamos la media en cada muestra bootstrap.
3. Observamos la dispersión de esas medias.

Si casi todas las medias están cerca de 50, la media estimada es estable.

Si aparecen medias bastante distintas, existe mayor incertidumbre.

Bootstrap puede emplearse para aproximar:

- el error estándar de una estimación;
- la variabilidad de una media, proporción o mediana;
- la variabilidad de parámetros estimados por un modelo;
- intervalos de confianza;
- la estabilidad de determinados procedimientos estadísticos o de aprendizaje automático.

Para este tema, la idea más importante es:

Bootstrap permite estudiar la incertidumbre de una estimación cuando solo tenemos una muestra observada.

Una posible trampa conceptual es confundir el valor real con el valor calculado en la muestra.

- Un **parámetro** es un valor desconocido de la población, por ejemplo su media real.
- Un **estimador** es el procedimiento que usamos para aproximarlos, por ejemplo calcular la media muestral.
- Una **estimación** es el valor concreto obtenido al aplicar el estimador a nuestros datos.

Bootstrap trabaja repitiendo artificialmente el cálculo del estimador sobre remuestras para observar cómo podría variar la estimación.

B representa el número de muestras bootstrap generadas.

- Si B es pequeño, la aproximación puede ser poco estable.
- Si B es grande, la estimación de la variabilidad suele ser más fiable.
- Usar un B grande aumenta el coste computacional, pero no implica disponer de más información original.

Ejemplo:

- $B = 10$: pocas remuestras, aproximación rudimentaria.
- $B = 1000$: aproximación mucho más habitual y estable.

EXAM=AAII.EX.2022S0.2

Jackknife es una técnica de remuestreo (*resampling*) estadístico determinista que permite mejorar la estimación del sesgo. A diferencia de bootstrap, no es aleatorio, por lo que en cada ejecución se obtendrán los mismos resultados.

No entra dentro del temario de la asignatura, pero en algún examen se ha ofrecido su definición como distractor en preguntas tests sobre la definición de bootstrap.

Agregación de Bootstrap: Bagging

Bibliografía:

- [ISL 8.2.1]
- [ESL 8.7] Pags. 282-288
- [ESL 8.7.1]
- Bloc de notas interactivo “Bagging” de la asignatura.
- “Data Transform Bagging”

EXAM=(2023J2.3,2025J2.3,AAII.EX.2025J2.9,2025S0.8,2025S0.10)

Bagging viene de **Bootstrap Aggregation**:

- **Bootstrap**: se generan muchas muestras aleatorias con reemplazo a partir del conjunto de entrenamiento.

- **Aggregation:** se entrenan modelos sobre esas muestras y se combinan sus predicciones.

EXAM=(2023J2.1,2025J2.2)

Cuando hay una función pérdida cuadrática, la idea teórica clave es que **promediar predicciones puede reducir la varianza sin modificar el sesgo**. Por eso se dice de manera general que el **objetivo principal** de bagging es **reducir la varianza** del modelo agregado, manteniendo aproximadamente el mismo sesgo.

EXAM=(2026J2.6)

Bagging es especialmente útil con modelos inestables o de alta varianza, como los árboles de decisión: pequeños cambios en los datos pueden producir árboles muy diferentes.

Partimos de un conjunto de entrenamiento original.

1. Generar B muestras bootstrap del conjunto de entrenamiento.
2. Entrenar un modelo independiente sobre cada muestra bootstrap.
3. Obtener la predicción de cada modelo para una nueva observación.
4. Agregar las B predicciones.

En regresión, se promedian las predicciones:

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$

En clasificación, la forma habitual y más sencilla es usar **votación mayoritaria**:

La clase predicha por el conjunto es la clase más votada por los modelos individuales.

B es el **número de modelos** entrenados, por ejemplo, el número de árboles cuando se aplica bagging con árboles.

- Un B mayor proporciona más predicciones que agregar.
- Aumentar mucho B incrementa el coste computacional.
- En bagging con árboles, usar muchos árboles no suele provocar sobreajuste adicional importante; se elige un valor suficientemente grande para que el error se estabilice.

Un árbol individual puede variar mucho si cambia el conjunto de entrenamiento. Bagging entrena muchos árboles con pequeñas perturbaciones aleatorias de los datos y promedia sus resultados.

La intuición es la misma que al promediar medidas ruidosas:

Las peculiaridades o errores inestables de unos modelos pueden compensarse con los de otros.

Si los modelos no cometen exactamente los mismos errores, el promedio es más estable que cualquiera de ellos por separado.

EXAM=(2021J2.20)

Con pérdida cuadrática, la idea teórica que debe memorizarse es que **promediar predicciones reduce la varianza y deja el sesgo sin cambios**.

EXAM=(2022J2.1)

Bagging funciona mejor cuando los modelos individuales tienen errores poco correlacionados.

- Si todos los árboles son casi idénticos, tenderán a equivocarse en los mismos casos.
- Si sus errores son distintos, el promedio o la votación puede compensar fallos individuales.
- Cuanto más correlacionados estén los modelos, menor será la reducción de varianza obtenida al agregarlos.

**Matiz importante para una pregunta de examen:—>

En una pregunta de 2022 aparece como correcta la opción que habla de modelos que no estén correlados mediante perturbaciones aleatorias que “incrementen la varianza”. La idea que conviene memorizar correctamente es:

Las perturbaciones bootstrap generan modelos individuales diferentes, típicamente de alta varianza; al agregarlos, se busca reducir la varianza de la predicción final.

Los árboles de decisión son un caso especialmente adecuado para bagging porque suelen tener **alta varianza**:

- Un pequeño cambio en los datos puede cambiar las particiones del árbol.
- Dos muestras bootstrap pueden producir árboles distintos.
- Al promediar o votar muchos árboles, la predicción final es más estable.

En bagging, los árboles individuales suelen construirse:

- profundos;
- sin poda (**unpruned**);
- con sesgo bajo;
- con varianza alta de forma individual.

La lógica es:

No intentamos que cada árbol individual sea estable; permitimos árboles muy ajustados y reducimos su varianza al combinarlos.

EXAM=(2025SO.7,2025SO.9)

Podar un árbol suele simplificarlo, lo que puede aumentar su sesgo.

En bagging interesa que cada árbol individual pueda capturar bien la estructura de sus datos de entrenamiento:

- Un árbol grande y sin poda suele tener sesgo bajo.
- A cambio, tiene varianza alta.
- Bagging está precisamente diseñado para reducir esa varianza mediante la agregación.

Por eso, en bagging con árboles:

Se suelen usar árboles profundos y sin poda para mantener bajo el sesgo, confiando en el promedio o la votación para reducir la varianza.

Un árbol profundo por sí solo puede sobreajustar fácilmente: tiene baja tendencia al error sistemático, pero responde demasiado a los detalles de sus datos.

Sin embargo, en bagging:

- cada árbol se entrena con una muestra bootstrap distinta;
- cada árbol puede sobreajustar detalles diferentes;
- al agregar las predicciones, buena parte de esa inestabilidad se compensa.

Por tanto:

Los árboles grandes tienen menos sesgo y más varianza individual; bagging reduce esa varianza al agregarlos, por lo que el conjunto no aumenta el sobreajuste de forma significativa al añadir más árboles.

No debe confundirse con que bootstrap “corrija el sobreajuste” de cada árbol: los árboles individuales siguen siendo inestables, pero se mejora la predicción agregada.

Problema	Predicción final en bagging
Regresión	Media de las predicciones de los modelos
Clasificación	Clase más votada por los modelos

Ejemplo de clasificación:

Si cinco árboles predican:

- azul
- azul
- rojo
- azul

- rojo

la predicción final es **azul**, porque es la clase mayoritaria.

Elemento	Sesgo	Varianza
Árbol profundo individual	Bajo	Alta
Conjunto bagging de árboles profundos	Aproximadamente el mismo sesgo bajo	Menor varianza

La idea central de los exámenes es:

Bagging mejora las predicciones reduciendo la varianza y dejando el sesgo esencialmente sin cambios.

Esto se formula de manera especialmente clara bajo pérdida basada en error cuadrático.

En regresión con pérdida cuadrática, la explicación sesgo-varianza del bagging es directa: el promedio reduce la varianza sin cambiar el sesgo.

En clasificación con pérdida 0-1, la explicación matemática no es tan directa:

- bagging suele mejorar clasificadores inestables como árboles;
- pero no existe la misma garantía general de mejora por simple promediado.

Para el test, salvo que se pregunte específicamente por esta diferencia, debe retenerse la idea operativa:

En clasificación, bagging combina los árboles mediante votación mayoritaria y suele mejorar la estabilidad de árboles de decisión.

Cada árbol de bagging se entrena con una muestra bootstrap del conjunto de entrenamiento. Como el muestreo se realiza con reemplazo, algunas observaciones aparecen repetidas y otras no aparecen en la muestra usada para construir un árbol.

Las observaciones que no han sido utilizadas para entrenar un árbol concreto se llaman observaciones **OOB** (**out-of-bag**, fuera de bolsa) para ese árbol.

EXAM=(2023J2.3)

En promedio, una muestra bootstrap de tamaño N deja fuera aproximadamente el 36.8% de las observaciones originales, es decir, aproximadamente un tercio:

$$\left(1 - \frac{1}{N}\right)^N \rightarrow e^{-1} \approx 0.368$$

Por tanto:

- aproximadamente dos tercios de los datos aparecen al menos una vez en la muestra bootstrap de cada árbol;
- aproximadamente un tercio queda fuera y puede emplearse para evaluarlo.

Para estimar el error OOB del modelo bagging:

1. Para cada observación del conjunto de entrenamiento, se seleccionan solo los árboles para los que esa observación fue OOB.
2. Se obtiene una predicción usando únicamente esos árboles.
3. En regresión, se promedian sus predicciones.
4. En clasificación, se toma la clase más votada.
5. Se compara la predicción OOB de cada observación con su valor real.
6. Se calcula el error total: MSE en regresión o tasa de clasificación incorrecta en clasificación.

La idea clave es:

Cada observación se evalúa utilizando únicamente árboles que no fueron entrenados con ella.

Por eso el error OOB permite aproximar el error sobre datos nuevos sin reservar un conjunto de validación separado.

EXAM=(2021J2.13,2024J2.2)

El error OOB es una estimación interna del error de generalización del conjunto bagging.

Concepto	Idea
Error de entrenamiento	Evalúa con datos usados para entrenar; puede ser demasiado optimista
Error de validación o test	Evalúa con datos reservados que no se usaron para entrenar
Error OOB	Evalúa cada observación solo con los árboles que no la usaron durante su entrenamiento

Ventajas del error OOB:

- aprovecha el propio proceso bootstrap;

- no requiere separar un conjunto de validación adicional;
- evita realizar una validación cruzada separada;
- con suficientes árboles, proporciona una estimación muy próxima al error de test.

EXAM=(2022J2.3,2024S0.1)

En las preguntas de la asignatura aparece como correcta la idea de que el error OOB puede ser algo mayor que el error real del conjunto completo.

La intuición es:

- para predecir una observación mediante OOB solo se utilizan los árboles en los que quedó fuera;
- esos árboles son aproximadamente un tercio del bosque;
- emplear un menos árboles en la votación, la predicción es ligeramente menos precisa, lo que hace que aumente su error.
- la predicción final del modelo completo utiliza todos los árboles y puede ser ligeramente mejor.

Por tanto, para el test:

El error OOB tiende a ser una estimación algo pesimista del rendimiento del bosque completo, aunque con suficientes árboles es una buena estimación del error de generalización.

Un único árbol es fácil de interpretar mirando sus divisiones. En cambio, cuando se agregan muchos árboles, ya no existe un solo árbol representativo del modelo completo.

Bagging suele mejorar la precisión, pero pierde interpretabilidad directa.

Aun así, puede obtenerse una medida global de la **importancia de cada variable** observando cuánto mejora las divisiones de los árboles.

Problema	Medida utilizada para la importancia
Regresión	Disminución del RSS o suma de cuadrados residual
Clasificación	Disminución del índice de Gini

El procedimiento conceptual es:

1. Cada vez que una variable se usa para dividir un nodo, se registra cuánto mejora el criterio de división.
2. Se acumulan esas mejoras para cada variable.
3. Se promedian las contribuciones entre todos los árboles.
4. Una variable es más importante cuanto mayor sea la mejora total que produce.

En clasificación:

Una variable con gran importancia basada en Gini es aquella que, al utilizarse en los árboles, reduce mucho la impureza de los nodos.

En regresión:

Una variable con gran importancia basada en RSS es aquella que produce divisiones que reducen mucho el error cuadrático residual.

Subespacio aleatorio

Bibliografía:

- Básica
 - Vídeo “Random Subspace” de la asignatura.
 - Bloc de notas interactivo “Random Subspace” de la asignatura.

El método de **subespacio aleatorio** (en inglés, *random subspace*) es un método de ensemble en el que se entrenan varios modelos utilizando diferentes subconjuntos aleatorios de variables de entrada.

Si el conjunto original tiene muchas variables, cada subconjunto de variables define un **subespacio** del espacio de características original.

Ejemplo:

Modelo	Variables utilizadas
Modelo 1	D_1, D_2, D_3
Modelo 2	D_3, D_4, D_5
Modelo 3	D_5, D_8, D_9

Cada modelo realiza su predicción y, al final, las predicciones se agregan:

- en clasificación, normalmente mediante votación;
- en regresión, normalmente mediante promedio.

La idea principal es:

Al entrenar modelos con diferentes subconjuntos aleatorios de variables, se introducen diferencias entre ellos, lo que puede hacer que el conjunto sea más robusto que un único modelo.

Este método resulta especialmente adecuado para modelos sensibles a los datos de entrada, como los árboles de decisión, y puede ser útil cuando hay muchas variables.

Limitación importante:

Si hay muchas variables irrelevantes, algunos subespacios aleatorios pueden contener poca información útil y el rendimiento puede empeorar.

El bosque aleatorio incorpora la idea de aleatorizar variables, pero lo hace de manera distinta a subespacio aleatorio.

En subespacio aleatorio se selecciona un subconjunto de variables para entrenar cada modelo o árbol, mientras que en bosque aleatorio en cada nodo de cada árbol se selecciona un nuevo subconjunto aleatorio de variables candidatas para realizar la división.

Subespacio de selección de características

Bibliografía:

- Básica
 - Vídeo “Feature Selection Subspace” de prácticas de la asignatura.
 - Bloc de notas interactivo “Feature Selection Subspace” de la asignatura.
- Complementaria
 - [RSEFS]

El método de **subespacios mediante selección de características** (en inglés *feature selection subspace*) construye varios modelos sobre diferentes subconjuntos de variables de entrada, pero esos subconjuntos no se eligen al azar: se obtienen mediante técnicas de **selección de características** (*feature selection*).

La diferencia principal respecto a **Random Subspace** es:

Método	Cómo obtiene las variables de cada modelo
Random Subspace	Selecciona subconjuntos aleatorios de variables
Feature Selection Subspace	Selecciona subconjuntos según una métrica o método de relevancia

La selección de características intenta conservar las variables más útiles para predecir la variable objetivo y descartar variables irrelevantes o redundantes.

Algunos métodos posibles son:

- **Información mutua:** valora cuánto informa una variable sobre la variable objetivo.

- **RFE (Recursive Feature Elimination)**: entrena modelos, elimina progresivamente variables poco útiles y selecciona el subconjunto con mejor rendimiento.

Una vez obtenidos los subconjuntos de variables:

1. Se entrena un modelo con cada subconjunto seleccionado.
2. Cada modelo realiza su predicción.
3. Las predicciones se combinan mediante votación en clasificación o promedio en regresión.

Dos formas sencillas de construir el ensemble son:

- **Por tamaño del subconjunto**: crear un modelo con la mejor variable, otro con las dos mejores, otro con las tres mejores, etc.
- **Por distintos métodos de selección**: crear un modelo con las variables elegidas por información mutua, otro con las elegidas mediante RFE, etc.

La idea principal es:

Feature Selection Subspace introduce diversidad entrenando modelos con distintos subconjuntos de variables relevantes, mientras que Random Subspace introduce diversidad eligiendo las variables aleatoriamente.

No debe confundirse con Random Forest:

Método	Selección de variables
Feature Selection Subspace	Subconjuntos elegidos mediante criterios de relevancia para entrenar modelos
Random Forest	Subconjunto aleatorio de variables candidatas en cada nodo de cada árbol

Por tanto, una pregunta que indique que en cada nodo se selecciona un subconjunto **aleatorio** de características está preguntando por **Random Forest**, no por Feature Selection Subspace.

Bosques aleatorios

Bibliografía:

- Básica
 - [ESL] 15 “Random Forests”
 - Bloc de notas interactivo “Random forest” de la asignatura.
- Complementaria

– [RF]

Un **bosque aleatorio** (**Random Forest**) es un conjunto de árboles de decisión cuyas predicciones se agregan. Es una modificación de bagging diseñada para que los árboles sean menos parecidos entre sí.

La idea fundamental es:

Random Forest construye muchos árboles sobre muestras bootstrap y, además, en cada nodo limita la búsqueda de la mejor división a un subconjunto aleatorio de variables.

EXAM=(2026J2.9)

Con ello se pretende reducir la correlación entre los árboles y conseguir que la agregación reduzca mejor la varianza.

En bagging, cada árbol se entrena sobre una muestra bootstrap distinta, pero al dividir un nodo puede considerar todas las variables disponibles. Si existe una variable muy predictiva, muchos árboles tenderán a utilizarla en divisiones parecidas y acabarán siendo bastante similares.

EXAM=(2024J2.1, 2024S0.2)

El bosque aleatorio añade una perturbación adicional:

- cada árbol sigue entrenándose sobre una muestra bootstrap;
- en cada nodo se seleccionan aleatoriamente solo m variables candidatas;
- la mejor división se busca únicamente entre esas m variables.

Método	Muestra de entrenamiento de cada árbol	Variables consideradas en cada nodo
Bagging con árboles	Bootstrap	Normalmente todas las variables disponibles
Random Forest	Bootstrap	Un subconjunto aleatorio de m variables

Por tanto:

La diferencia esencial respecto a bagging es la selección aleatoria de variables candidatas en cada nodo.

Sea un conjunto de entrenamiento con p variables de entrada y sea B el número de árboles del bosque.

Para construir un Random Forest:

1. Repetir el proceso para $b = 1, \dots, B$.
2. Obtener una muestra bootstrap del conjunto de entrenamiento.
3. Construir un árbol sobre esa muestra.
4. En cada nodo del árbol, seleccionar aleatoriamente m variables de entre las p disponibles.
5. Buscar la mejor división solo entre esas m variables.
6. Continuar dividiendo el árbol, normalmente sin poda, hasta alcanzar el criterio de parada.
7. Agregar las predicciones de los B árboles.

Parámetro	Significado
p	Número total de variables de entrada disponibles
B	Número de árboles que forman el bosque
m	Número de variables seleccionadas aleatoriamente como candidatas en cada nodo

Para memorizar:

- B no representa variables: representa el **número de árboles**.
- m no representa el número total de variables del árbol: representa las **variables candidatas consideradas en cada división**.
- La selección de las m variables se repite en **cada nodo**, no se hace una única vez para todo el árbol.

EXAM=(2021J2.23)

Una ventaja del bosque aleatorio es que, al construir cada nodo, no se prueban todas las variables disponibles, sino solo un subconjunto aleatorio de m variables.

Esto reduce el coste de buscar divisiones respecto a bagging con árboles que consideran todas las variables en cada nodo.

Según la formulación teórica del algoritmo, la eficiencia de construcción es **lineal en el número total de variables de entrada**.

Idea de examen:

La eficiencia del algoritmo de construcción de bosques aleatorios es lineal en el número total de variables de entrada.

No debe confundirse con:

- coste cuadrático en m ;
- coste $O(m^p)$;

- “mayor eficiencia cuantos más árboles contenga”, porque más árboles normalmente aumentan el coste computacional.

La formulación estudiada de Random Forest selecciona un subconjunto aleatorio de características en cada nodo.

El procedimiento de división es:

1. Elegir aleatoriamente m variables de entre las p disponibles.
2. Evaluar posibles divisiones solo para esas m variables.
3. Elegir la mejor división según el criterio del árbol, por ejemplo Gini o entropía en clasificación.

No se seleccionan las variables:

- usando siempre todas las características;
- en función de su correlación previa con la variable objetivo;
- mediante la importancia por permutación;
- fijando necesariamente el mismo subconjunto para todo el árbol.

Idea de examen:

En Random Forest, las características para dividir en cada nodo se seleccionan formando un subconjunto aleatorio de características.

Como valores iniciales habituales:

Problema	Valor orientativo de m
Clasificación	$m \approx \sqrt{p}$
Regresión	$m \approx p/3$

Estos valores son orientativos: m es un parámetro que puede ajustarse según el problema.

Su efecto intuitivo es:

Valor de m	Efecto habitual
Grande, cercano a p	Árboles más parecidos; mayor correlación
Más pequeño	Árboles más diversos; menor correlación
Demasiado pequeño	Puede impedir que aparezcan variables útiles en algunos nodos

Caso importante:

Si $m = p$, cada nodo puede considerar todas las variables y Random Forest se aproxima a bagging de árboles.

EXAM=(2021J2.1)

Si el número total de variables p es grande, pero solo unas pocas variables son realmente relevantes, elegir un valor demasiado pequeño de m puede perjudicar al bosque.

La razón es que, en cada nodo, solo se consideran m variables candidatas. Si m es muy pequeño y hay pocas variables útiles, puede ocurrir que en muchos nodos no aparezca ninguna variable relevante entre las candidatas.

Consecuencia:

Con muchas variables irrelevantes y pocas relevantes, Random Forest puede funcionar mal para valores pequeños de m .

No significa que siempre convenga $m \ll p$:

- un m pequeño aumenta la diversidad entre árboles;
- pero si es demasiado pequeño, puede ocultar variables importantes en demasiadas divisiones.

Idea de examen:

Si p es grande y las variables relevantes son pocas, los bosques aleatorios pueden funcionar mal para valores pequeños de m .

Una vez contruidos los árboles, sus predicciones se agregan.

Problema	Predicción final
Regresión	Media de las predicciones de los árboles
Clasificación	Votación: se elige la clase más votada

En regresión:

$$\hat{f}_{RF}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

EXAM=(2022J2.2,2024S0.3)

En clasificación, cada árbol vota una clase y el bosque devuelve la clase con más votos.

Importante para el test:

En clasificación gana la **clase mayoritaria**, es decir, la más votada; no es necesario que obtenga mayoría absoluta.

Ejemplo con tres clases:

Clase	Votos
A	40
B	35
C	25

La predicción final es la clase **A**, aunque no supere el 50% de los votos.

EXAM=(2025S0.1)

Por qué interesa que los árboles sean diferentes:

Los árboles de decisión individuales suelen tener baja estabilidad: pequeñas variaciones en los datos pueden producir árboles distintos. La agregación aprovecha esta característica solo si los árboles no cometen todos los mismos errores.

- Si los árboles son diferentes, sus errores pueden compensarse al promediar o votar.
- Si los árboles son muy similares, tenderán a fallar en las mismas observaciones.
- En ese caso, la predicción agregada aporta menos mejora.

Bajo la intuición de regresión, si los árboles tienen varianza individual σ^2 y correlación aproximada ρ , la varianza del promedio depende de:

$$\text{Var}(\text{promedio}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

Al aumentar B , disminuye el segundo término, pero el término asociado a la correlación permanece.

Por eso:

Si los árboles son muy similares entre sí, la varianza del conjunto no disminuye lo suficiente.

Random Forest intenta evitarlo haciendo que distintos árboles y distintos nodos tengan acceso a variables candidatas diferentes.

Los árboles de Random Forest suelen crecer bastante y no podarse:

- un árbol profundo puede mantener un sesgo bajo;
- individualmente puede tener varianza alta;

- la agregación y la descorrelación buscan reducir la varianza del conjunto.

EXAM=(2022S0.1)

El parámetro B controla cuántos árboles se agregan.

- Al aumentar B , la predicción suele estabilizarse.
- Aumenta el coste computacional.
- Añadir árboles no suele provocar sobreajuste adicional por sí mismo.

La afirmación correcta es:

Un bosque no mejora porque cada árbol sea sencillo o esté muy podado, sino porque combina muchos árboles suficientemente buenos y poco correlacionados.

En clasificación, los árboles necesitan un criterio para elegir la mejor división de un nodo. Entre los criterios posibles están:

- índice de Gini;
- entropía.

EXAM=(2025J2.6)

El criterio de división (por ejemplo, Gini o entropía) determina cómo se eligen los cortes en cada nodo. Cambiarlo puede producir árboles con estructuras algo diferentes y modificar ligeramente la diversidad y el rendimiento del bosque.

Por tanto, elegir entropía en lugar de Gini puede afectar a los splits y generar árboles algo diferentes o variados.

No implica automáticamente:

- reducir el número de árboles necesarios;
- mejorar la interpretación de las reglas;
- obligar a utilizar validación cruzada en cada árbol.

Importancia de variables en bosques aleatorios:

Al tener muchos árboles, un Random Forest no se interpreta observando un único árbol. En su lugar, puede resumirse qué variables resultan más importantes para las predicciones del bosque.

En el temario aparecen dos medidas principales:

Medida	Idea
Importancia por permutación aleatoria	Una variable es importante si desordenar sus valores empeora mucho el rendimiento

Medida	Idea
Importancia por reducción de Gini	Una variable es importante si sus splits reducen mucho la impureza en los árboles

EXAM=(2023S0.2,2025J2.4)

Importancia por permutación OOB:

La importancia por permutación utiliza las observaciones OOB, ya estudiadas en bagging.

Para una variable X_j :

1. Se calcula el rendimiento del árbol sobre sus observaciones OOB.
2. Se permutan aleatoriamente los valores de X_j en esas observaciones OOB.
3. Se vuelve a calcular el rendimiento con la variable desordenada.
4. Se mide cuánto empeora la precisión o cuánto aumenta el error.
5. Se promedia ese deterioro entre todos los árboles.

La permutación rompe la información predictiva aportada por esa variable sin modificar las demás.

Interpretación:

- Si al permutar una variable la precisión empeora mucho, la variable es importante.
- Si el rendimiento apenas cambia, la variable tiene poca importancia para la predicción.

Idea de examen:

La importancia por permutación se calcula permutando los valores de cada variable en los datos OOB y promediando el deterioro de la precisión.

EXAM=(2025J2.4)

Importancia basada en Gini:

En clasificación, cada split puede reducir la impureza del nodo según el índice de Gini.

Para medir la importancia de una variable:

1. Se registran las reducciones de Gini producidas por los splits que utilizan esa variable.
2. Se acumulan estas reducciones a lo largo de los árboles.

3. Cuanto mayor sea la reducción total o media, mayor importancia se atribuye a la variable.

Interpretación:

Una variable importante según Gini es una variable que suele generar particiones útiles en los árboles del bosque.

Técnica	¿Mide importancia de variables en Random Forest?	Motivo
Reducción del índice de Gini	Sí	Mide cuánto mejoran los splits asociados a una variable
Permutación aleatoria de variables en OOB	Sí	Mide cuánto empeora la predicción al inutilizar una variable
Matriz de confusión	No	Evalúa errores de clasificación, no la contribución de cada variable
Prueba chi-cuadrado	No es la técnica estándar preguntada	No corresponde a las medidas propias del bosque indicadas en el temario
Regularización por variable	No	No es el mecanismo de importancia de Random Forest

Random Forest mejora bagging introduciendo selección aleatoria de m variables en cada nodo: construye B árboles menos correlacionados, predice por media o votación mayoritaria y mide la importancia mediante reducción de Gini o deterioro al permutar variables en OOB.

ExtraTree

Bibliografía:

- Básica
 - Vídeo “ExtraTrees” de prácticas de la asignatura.
 - Bloc de notas interactivo “ExtraTrees” de la asignatura.

Un **árbol extremadamente aleatorio**, más comúnmente conocido como **ExtraTree** (acrónimo de *Extremely Randomized Tree*) es un método de ensemble formado por muchos árboles de decisión, normalmente profundos y sin poda.

Está relacionado con bagging y bosque aleatorio, pero introduce todavía más aleatoriedad al construir los árboles.

Las predicciones se agregan igual que en otros conjuntos de árboles:

Problema	Predicción final
Regresión	Media de las predicciones de los árboles
Clasificación	Votación mayoritaria

La diferencia principal está en cómo se construyen las divisiones de cada árbol:

Método	Datos usados por cada árbol	Variables candidatas en cada nodo	Punto de división
Random Forest	Normalmente una muestra bootstrap	Subconjunto aleatorio	Se busca el mejor corte entre las candidatas
Extra Trees	Habitualmente todo el conjunto de entrenamiento	Subconjunto aleatorio	Se prueban cortes aleatorios y se elige entre ellos

Por tanto:

Random Forest aleatoriza las variables candidatas; Extra Trees aleatoriza además los puntos de corte.

Esta aleatoriedad adicional hace que los árboles sean menos parecidos entre sí, lo que puede beneficiar la agregación, aunque cada árbol individual puede ser algo menos preciso.

Los parámetros más importantes son:

- número de árboles del conjunto;
- número de variables candidatas consideradas en cada nodo;
- tamaño mínimo necesario para dividir un nodo o para formar una hoja.

Extra Trees es una variante muy aleatorizada de los conjuntos de árboles: construye muchos árboles, selecciona variables aleatoriamente en los nodos y utiliza divisiones aleatorias; después combina sus predicciones por media o votación.

Data Transform Bagging

Bibliografía:

- Básica
 - Vídeo “Data Transform Bagging” de prácticas de la asignatura.
 - Bloc de notas interactivo “Data Transform Bagging” de prácticas de la asignatura.

Bagging mediante transformaciones de datos, más conocida como ***Data Transform Bagging*** y a veces ***Data Transform Ensemble***, es un método de ensemble en el que se entrenan varios modelos sobre distintas versiones transformadas del mismo conjunto de entrenamiento.

En bagging clásico, la diversidad se obtiene mediante diferentes muestras bootstrap. En este enfoque, la diversidad se obtiene aplicando distintas transformaciones a las variables de entrada.

La idea principal es:

Si cada modelo observa una representación algo diferente de los datos, sus predicciones o errores pueden estar menos correlacionados y la agregación puede mejorar el resultado final.

El procedimiento general es:

1. Partir del mismo conjunto de entrenamiento.
2. Aplicar diferentes transformaciones de datos para obtener varias versiones del mismo conjunto.
3. Entrenar un modelo sobre cada versión transformada.
4. Combinar las predicciones de los modelos.

La agregación suele realizarse mediante:

Problema	Predicción final
Regresión	Media de las predicciones
Clasificación	Votación mayoritaria o combinación de probabilidades

Algunas transformaciones habituales son:

- normalización a un rango fijo;
- estandarización, con media cero y escala comparable;
- escalado robusto frente a valores atípicos;
- transformaciones de potencia para reducir asimetrías;
- transformación por cuantiles;
- discretización en intervalos (**k-bins**).

Este enfoque es más útil cuando el modelo base es sensible a la escala o a la distribución de las variables de entrada, por ejemplo:

- regresión logística;
- redes neuronales;
- KNN;
- máquinas de vectores de soporte.

En cambio, los árboles de decisión suelen ser poco sensibles a simples cambios de escala, por lo que no siempre obtendrán tanta diversidad únicamente normalizando o estandarizando los datos.

Método	De dónde obtiene la diversidad
Bagging clásico	Diferentes muestras bootstrap de filas
Random Subspace	Diferentes subconjuntos aleatorios de columnas
Feature Selection Subspace	Diferentes subconjuntos de columnas elegidos mediante criterios de relevancia
Data Transform Ensemble	Diferentes transformaciones de las variables de entrada

Idea para recordar:

Data Transform Ensemble combina modelos entrenados sobre diferentes representaciones transformadas de los mismos datos, buscando diversidad entre predicciones para obtener un conjunto más robusto.

Recursos

Yggdrasil Decision Forest (YDF) es una biblioteca desarrollada por Google.

- Documentación YDF
- Repositorio de YDF en GitHub

Bibliografía

Bibliografía:

- Básica
 - [ISL] An introduction to statistical learning, G. James, D. Witten, T. Hastie and R. Tibshirani: <https://www.statlearning.com/>
 - [ESL]
 - [URF] Understanding Random Forests: from Theory to Practice, Gilles Loupe, Univ. de Liège: <https://arxiv.org/abs/1407.7502>
 - [WIB] What Is Bootstrapping in Statistics? <https://www.thoughtco.com/what-is-bootstrapping-in-statistics-3126172>
- Complementaria
 - [RF] Random Forests. Breiman, L. Machine Learning (2001) 45: 5. <https://doi.org/10.1023/A:1010933404324>

- Decision Tree Cheat Sheet
- [RFyO] Random Forest y OOB <https://cienciadedatos.net/documentos/py36-grid-search-random-forest-out-of-bag-error-early-stopping>
- [CyRF] CART y Random Forest <https://bookdown.org/content/2031/>
- [RFA] Random Forest Algorithm <https://www.simplilearn.com/tutorials/machine-learning-tutorial/random-forest-algorithm>
- [RSEFS] TRAN, Cao Truong. Random Subspace Evolutionary Feature Selection for High-dimensional Data. Research Square, 2023. <https://www.researchsquare.com/article/rs-3229911/v1>

FAQ:

- FAQ

Licencia y atribución

License CC BY 4.0

Este trabajo está licenciado bajo la licencia **Creative Commons Atribución 4.0 Internacional**.

© 2026, Colaboradores de apuntes-muicd-uned.